

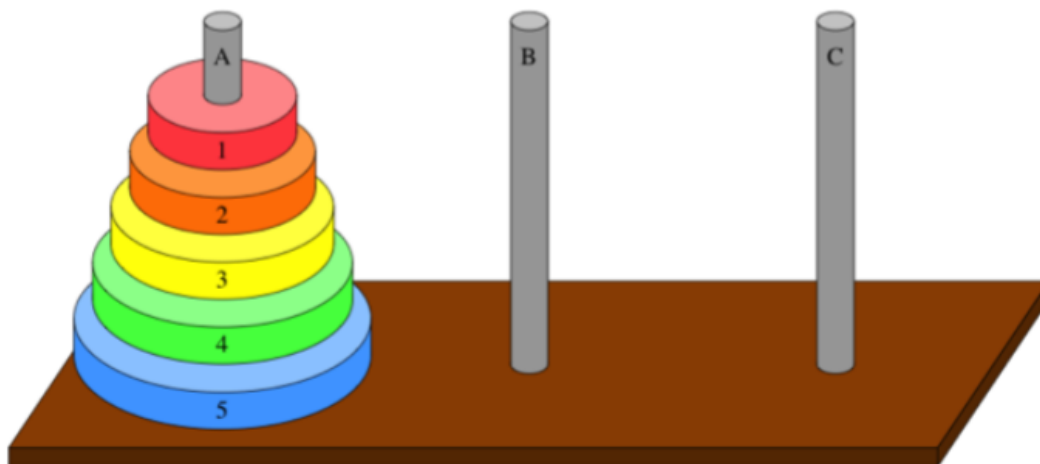
Tower of Hanoi

Arman Bryan, Zain Khan, Wren

The program takes an input of N disks. It moves all disks from the source rod onto a final rod using another rod as a helper. Using recursion, it first calls itself, with that function call calling itself, and that subsequent function call calling itself, and so on, until $N \leq 0$, to move the top $N-1$ disks out of the way and onto the helper rod.

Once each sub-call is finished, it then “pops” the exposed disk off the top of the initial rod and then “appends” it to the destination list. This is a stack data structure, so each new element is first-in-last-out, just like the Tower of Hanoi problem. The fact that it’s recursive (move $N-1$ out of the way before moving disk N) means that it will never make an “illegal” move (placing a larger disk on a smaller one). Each of the recursive function calls leaves that largest disk remaining on the initial rod. The function then performs just one move, moving that disk from the initial rod to the destination.

Once that move is complete, it then calls itself again using recursion, but this time it’s moving from the helper to the destination, with each function call calling itself recursively, but this time it’s from the helper rod to the destination rod, and the initial rod is now acting as the helper rod. Each function call moves the disk into its final and correct position on the destination rod. The recursion terminates when a call is made for $N=0$ disks.



A: [3, 2, 1]
B: []
C: []

Move disk 1 from A to C

A: [3, 2]
B: []
C: [1]

Move disk 2 from A to B

A: [3]
B: [2]
C: [1]

Move disk 1 from C to B

A: [3]
B: [2, 1]
C: []

Move disk 3 from A to C

A: []
B: [2, 1]
C: [3]

Move disk 1 from B to A

A: [1]
B: [2]
C: [3]

Move disk 2 from B to C

A: [1]
B: []
C: [3, 2]

Move disk 1 from A to C

A: []
B: []
C: [3, 2, 1]

Tower of Hanoi completed!